mhs-poisson-z: implementation

Technical notes on `run.mode = "mhs-poisson-z"`. HSEMHS, 24 Oct 2026.

1. Running it

```
[mpiexec -n R] julia --project=. -t T scripts/run.jl config/modes/mhs-poisson-z.toml
```

Driver `run_mhs_poisson_z` (`src/RunModes.jl`), solver `module PoissonZ` (`src/PoissonZ.jl`), MPI layer `src/Slab.jl`. The input and the `[column_hse]`, `[tau]` keys are those of `column-hse-z`; in addition:

Key	Default	Meaning
`poisson.z_cut_m`	10^6	solved region $z \le z_{\rm cut}$; column HSE above
`poisson.max_steps`	100	Gauss–Newton steps
`poisson.rtol`, `n_consec`	10^{-4}, 3	stop when $\Delta\Phi/\Phi <$ rtol on `n_consec` consecutive steps
`poisson.cg_tol`, `cg_maxit`	10^{-3}, 80	PCG relative residual and iteration cap per step
`poisson.anchor_w`	1	weight of the mean-ln P anchor (bottom plane)

2. Flow

1. `open_input`, `slab_par` (x slabs), `read_field` of T; `column_hse` (the `column-hse-z` integrator) on the same decomposition gives P_0.
2. $\mathbf{B}$ read; all fields converted to (z, y, x) slab arrays; `PoissonZ.lorentz`: $\mathbf{J} = \nabla \times \mathbf{B}/\mu_0$ with centred periodic x, y differences (halo exchange in x) and `np.gradient`-type z differences, $\mathbf{L} = \mathbf{J} \times \mathbf{B}$ at cell centres, fixed.
3. `solve_pressure`: Gauss–Newton on the layers $z \le z_{\rm cut}$ (below).
4. Above $z_{\rm cut}$: per column, trapezoidal hydrostatic step upward, $u_k - u_{k+1} + \frac{\Delta z\, g}{2}(q_k + q_{k+1}) = 0$ ($q = \rho/P$), solved by Newton with $\partial q/\partial u = q(\partial \ln\rho/\partial \ln P - 1)$.
5. `tau_columns`: ρ, χ_{500}, τ_{500} (`exp_tail`), $z(\tau{=}1)$; cooperative output.

3. Discretisation and objective

Unknown $u = \ln P$ on the n solved layers. With $e = e^{-u}$, $q = \rho/P$ from the EOS:

$$R_x = -D_x u + L_x e, \qquad R_y = -D_y u + L_y e \qquad \text{(centres)},$$

$$R_{z,f} = -\frac{u_f - u_{f+1}}{h_f} - g\frac{q_f + q_{f+1}}{2} + \frac{L_{z,f} e_f + L_{z,f+1} e_{f+1}}{2} \qquad \text{(interior faces)},$$

D_x, D_y centred periodic. $\Phi = \sum w_k \bar H_k^2 (R_x^2 + R_y^2) + \sum h_f \bar H_f^2 R_{z,f}^2$, w cell heights, $\bar H_k$ the global layer median of $1/(gq)$ in the start state, $\bar H_f = \sqrt{\bar H_k \bar H_{k+1}}$ (weights fixed).

4. Gauss–Newton step

Linearisation: $\delta R_x = -D_x \delta u - L_x e\, \delta u$ (same for y), $\delta R_{z,f} = -(a_f \delta u_f + b_f \delta u_{f+1})$ with $a_f = 1/h_f + \frac{1}{2}(g\, q'_f + L_{z,f} e_f)$, $b_f = -1/h_f + \frac{1}{2}(g\, q'_{f+1} + L_{z,f+1} e_{f+1})$, $q' = \partial q/\partial u$. Normal equations $(J^\top W J + E + A)\, \delta u = -J^\top W \mathbf{R}$, matrix free (`Aop!`, `JtW!`):

- $E = 10^{-2} \kappa^2 w \bar H^2$ damps the modes centred differences cannot see (checkerboard), $\kappa = 2\pi/L_x$;
- anchor $A = A_{\rm anc}\, \mathbf{c}\mathbf{c}^\top$, $\mathbf{c}$ the indicator of the bottom plane, a rank-one *penalty* on the bottom-plane mean of δu, $A_{\rm anc} =$ `anchor_w`$\cdot w_{f,n-1}/h_{n-1}^2 / \sum c$. It keeps that mean near its starting value but does not fix it exactly, and the element-wise clamp below can shift it further (unlike `full-mhs-z`, whose gauge is an exact reparameterisation).

Step: PCG to `cg_tol` or `cg_maxit`; $|\delta u| \le 2$ per node (clamp); backtracking on Φ (halving, up to 12 times, with the EOS status checked).

Termination. The iteration ends when (a) the relative decrease $(\Phi_{\rm prev} - \Phi)/\Phi_{\rm prev}$ of accepted steps is below `rtol` on `n_consec` consecutive steps (log line `converged:  ...`); (b) no halving gives a decrease (`no decrease along the step ...; stopping`); or (c) `max_steps` is reached (no message). The driver returns `converged`, i.e. exit 0, in all three cases: **exit 0 is not evidence of convergence**; read the log (code follow-up: propagate the termination into the status). Even case (a) does not certify a small force residual.

5. Preconditioner

The Laplacian part with layer-median coefficients: in horizontal Fourier space, mode (k_x, k_y) with $s^2 = \sin^2(k_x\Delta x)/\Delta x^2 + \sin^2(k_y\Delta y)/\Delta y^2$, the vertical tridiagonal system $\mathrm{diag}(c_k + w_k \bar{H}_k^2 s^2 + E_k)$ with off-diagonals from the medians of a_f, b_f (`setprec!`, exact global medians by `gmedians`), solved by the Thomas algorithm (`thomas!`). The rank-1 anchor acts only on the mean mode $(0, 0)$ and is added there exactly by Sherman–Morrison. Without it, CG stalled at its cap after the anchor was changed from the bottom plane to an irregular cell set; with it, 9–80 iterations.

Distributed FFT (`SlabFFT`): (1) per local x plane a real FFT along y (`plan_rfft` on contiguous (n, N_y) slices, threaded over x); (2) `MPI.Alltoallv` transpose from x slabs to slabs of k_y (each rank sends $(n,\ k_y \in \text{dest},\ x \in \text{own})$ blocks); (3) complex FFT along x; the vertical solve then runs on contiguous columns $T[:, k_x, j_\ell]$, threaded over local k_y; the inverse retraces the steps. FFTW's own threading is not used (it segfaults under Julia 1.13). With one rank the transpose is a local permutation.

6. Parallelisation and reproducibility

- x slabs (`slab_par`); y and z complete on every rank. Halo planes in x for every D_x (`sddx!`, `MPI.Sendrecv!`); all reductions global (`gsum`, `gdot`, `gnorm`, `gmax`); medians exact (`gquantiles`, bisection on order-preserving integer keys).
- Threads in every loop over x (stencils, EOS calls, Thomas solves, CG vector updates via `tbcast!`; plain broadcasts and masked indexing were serial bottlenecks: the anchor's `v[anc]` allocated 280 MB per operator call).
- Reproducibility: medians and quantiles are exact global order statistics, but sums and dot products are reduced in a rank-dependent order, so results agree only to rounding, and an iterative solver can amplify that along its path; identical trajectories for different rank or thread counts are not promised. Measured (manual, not an automated test): 96^2 crop, 1 vs 4 ranks, $|\Delta \ln P| \leq 1.5 \times 10^{-6}$.
- Threads: Julia threads in every loop; BLAS stays at 1 thread; FFTW plans single-threaded, applied from Julia threads.

7. Output

MHS checkpoint layout (read by `force_split.py`, `m5h_isotau*.py`): `/atmosphere/{Pgas, rho, T, tau500, Bx_centre, By_centre, Bz_centre, Pgas_column_hse, z_tau1_m}`, `/coords/{x_m, y_m, z_m}`; Julia (x, y, z), h5py (z, y, x), uncompressed; attributes `kind = poisson_step`, `z_cut_m`, `phi_start`, `phi_end`, `steps`, `P_ref_Pa`, `eos_table`, layout.

8. Verification and performance

- Against the original script (`scripts/poisson/poisson_step.jl` on the same column-HSE file, 96^2 crop): $|\Delta \ln P| \leq 5.8 \times 10^{-6}$.
- Full cube ($504 \times 504 \times 177$, $z_{\mathrm{cut}} = 1$ Mm, 77 solved layers from -0.49 to 0.98 Mm), 24 Oct 2026, commit 13666b7 (stop rule and step limit since changed), log `out/modes/mhs-poisson-z.log` in the cluster worktree `MHS_modes`; 8 ranks × 32 threads on 4 nodes: 4.7 min including column HSE, 4.1 GB per rank. Step 1 reduced Φ by 88 % (80 CG iterations, 18 s), step 2 by another 28 % (49 iterations); the run then used the then step limit (60), at $\Phi/\Phi_0 = 0.081$ and $\Delta\Phi/\Phi = 1.8 \times 10^{-4}$ per step with 9–11 CG iterations each.
- On a magnetic 96^2 crop Φ creeps ($\Delta\Phi/\Phi \approx 1.5 \times 10^{-3}$ per step for 100 steps): the low-β pressure inflation described in the physics document.